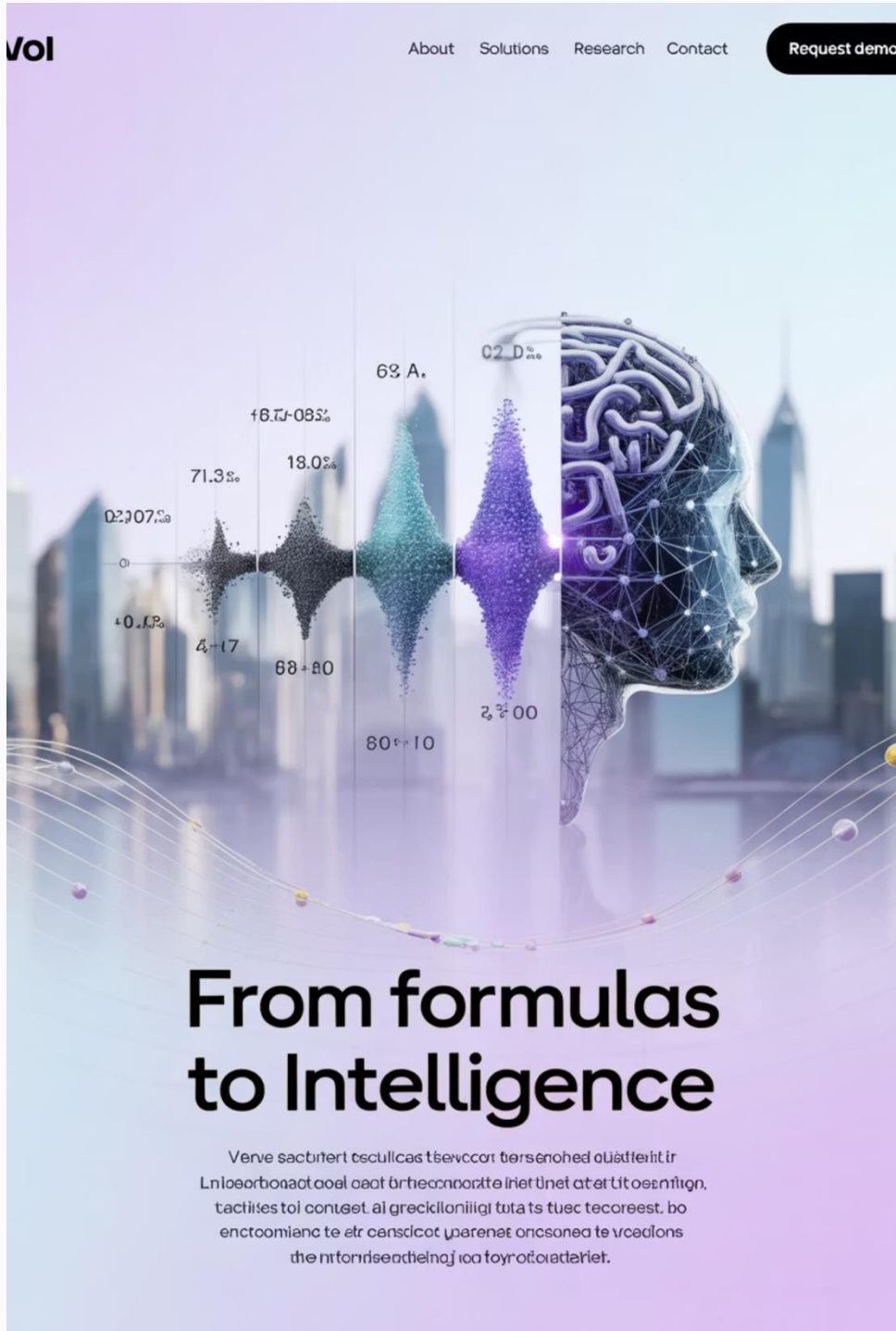
Prompt Engineering Local LLM

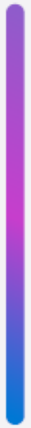
From mathematical models to generative AI

Andy Li, PhD, CFA, FRM



How does a Prompt work?

LLM sees prompt as sequence of tokens …



You can use the tool below to understand how a piece of text might be tokenized by a language model, and the total count of tokens in that piece of text.

It's important to note that the exact tokenization process varies between models. Newer models like GPT-3.5 and GPT-4 use a different tokenizer than our legacy GPT-3 and Codex models, and will produce different tokens for the same input text.

GPT-3.5 & GPT-4 | GPT-3 (Legacy)

I have a dream that my four little children will one day live in a nation where they will not be judged by the color of their skin but by the content of their character.

Clear | Show example

Tokens	Characters
36	169

I have a dream that my four little children will one day live in a nation where they will not be judged by the color of their skin but by the content of their character.

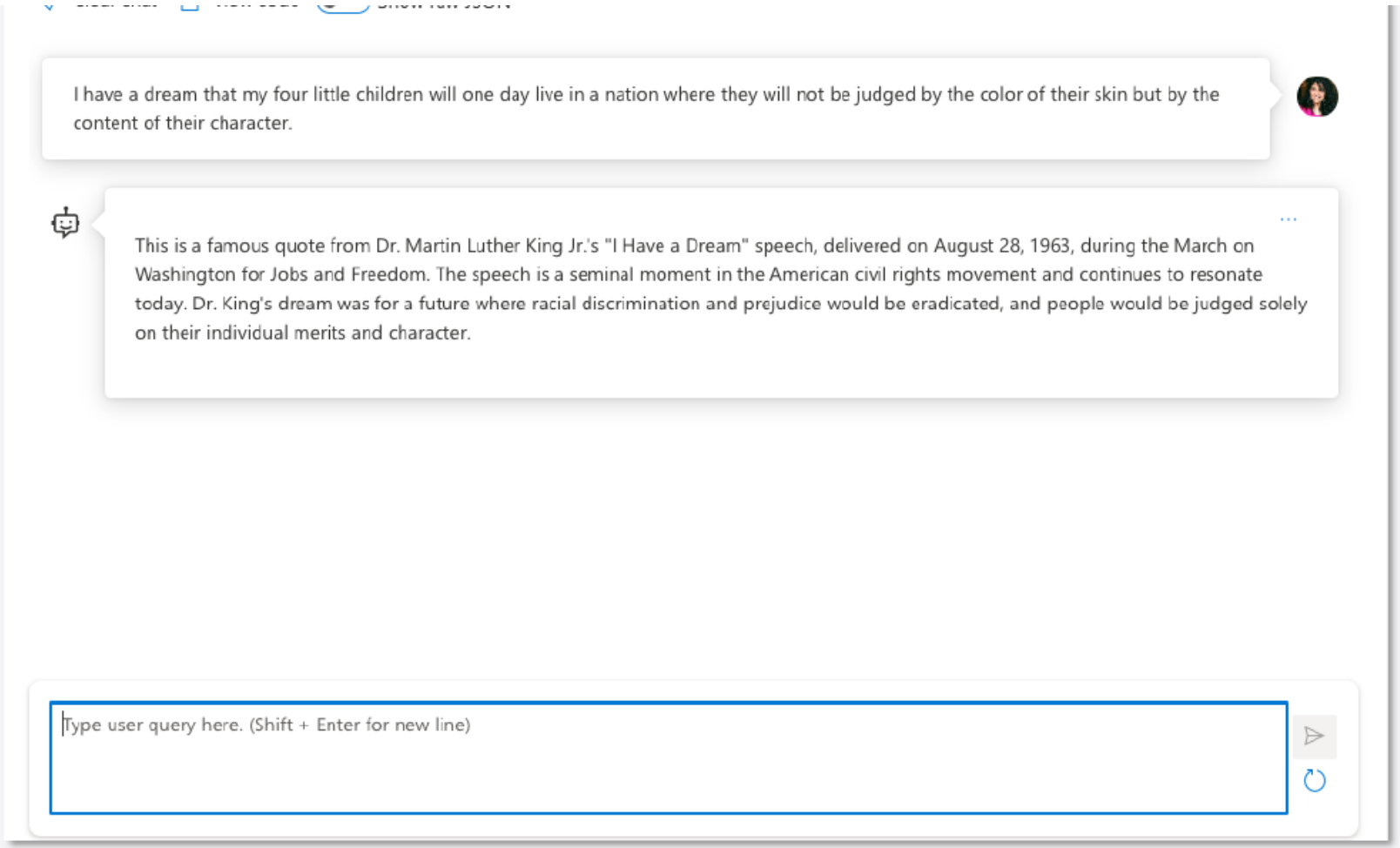
Try it yourself: https://platform.openai.com/tokenizer

Microsoft

How does a Prompt work?

Foundation LLMs will simply predict the next token

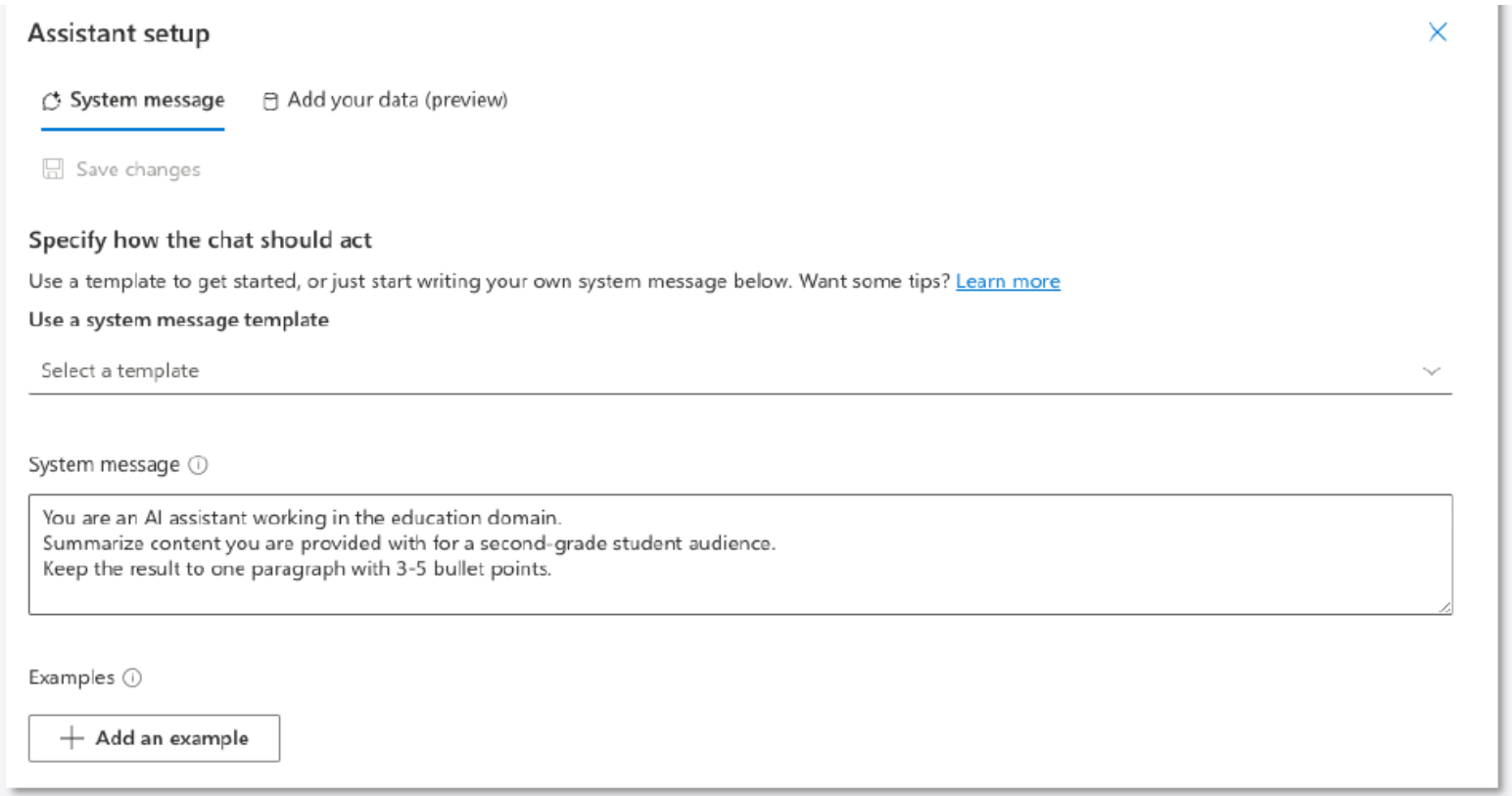
The user did not specify any instructions. AI treats this as request for information and predicts response from trained data.



How does a Prompt work?

Instruction-tuned LLM extends base behavior for task

Add context with a "system" message that tunes the default LLM behavior.



Definition

What is Prompt Engineering?

A prompt is the text input given to the Generative AI model.

Prompt engineering is the process of designing and optimizing the prompt till the response meets the user's expectations for relevance or quality

Motivation

What is Prompt Engineering Necessary?

- **LLMs are stochastic in nature**
  since prompts "program" the model, the responses are even more sensitive to construction of prompt.
- **LLMs can hallucinate responses**
  since pre-trained data has a cutoff date, they may predict completions that are not grounded in fact.
- **LLMs have diverse capabilities**
  every model has unique features and quirks that can be tuned for better quality responses

The Difference Between Prompt and Instruct

- **Prompt mode**:
  The input is usually a **word or short phrase**, and the model generates **natural and coherent text** based on this cue.
  This mode is well suited for **generative tasks**, such as text generation and document summarization.

- **Instruct mode**:
  The input is a **clear and explicit instruction** that asks the model to perform a **specific task**.
  This mode is suitable for tasks that require **direct guidance**, such as answering questions or explaining concepts.

C.R.A.F.T

🔍 **Context**: Provide background, details or sources in your prompt

🎭 **Role**: Specify who you want AI to “be” (e.g., expert, teacher, marketer)

👥 **Audience**: Specify who the response is for, better with details

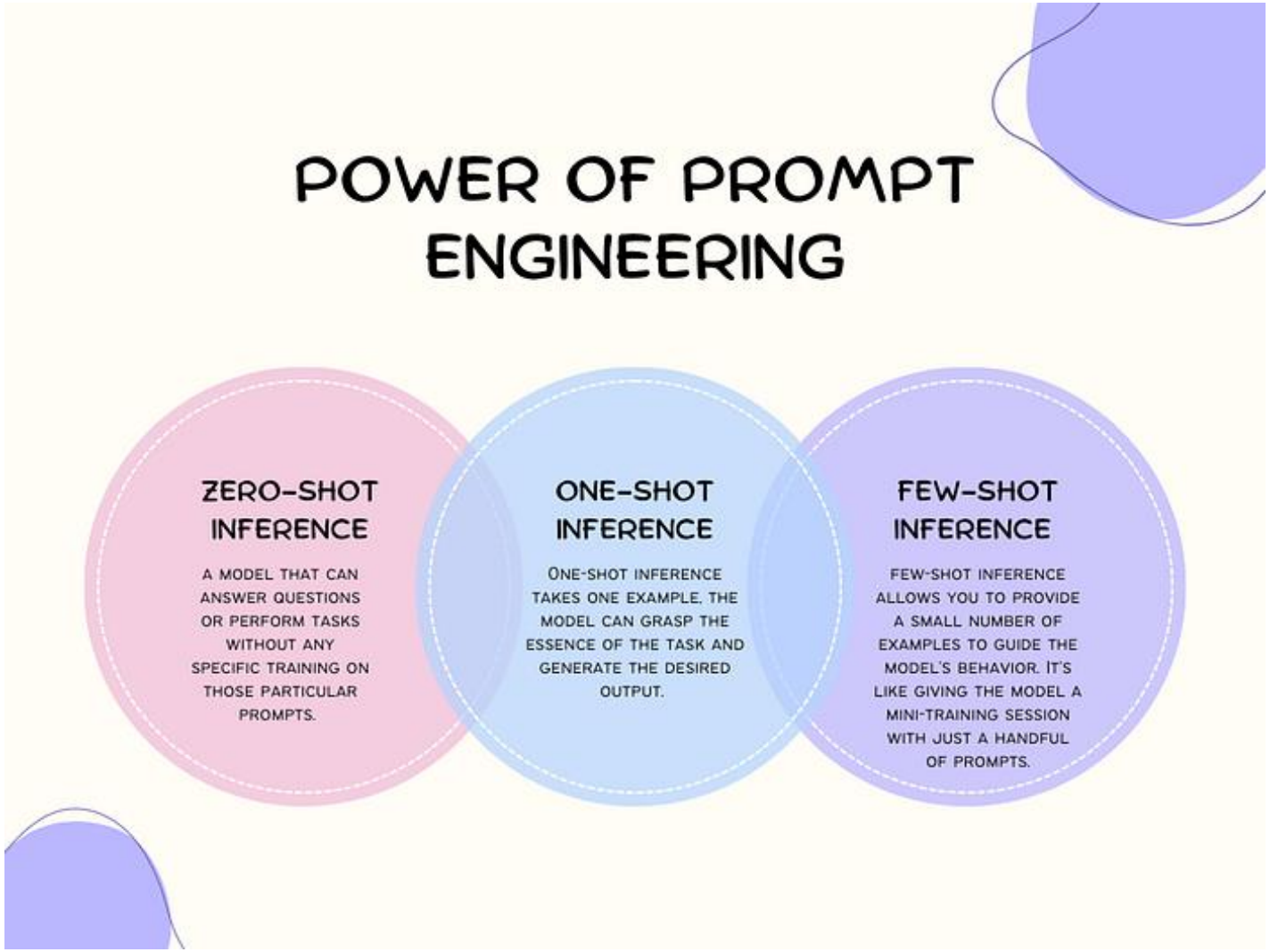
📝 **Format**: Decide on the output style, length, or structure (list, story, table, etc.)

💬 **Task & Tone**: Clearly state the desired action and the preferred style or mood



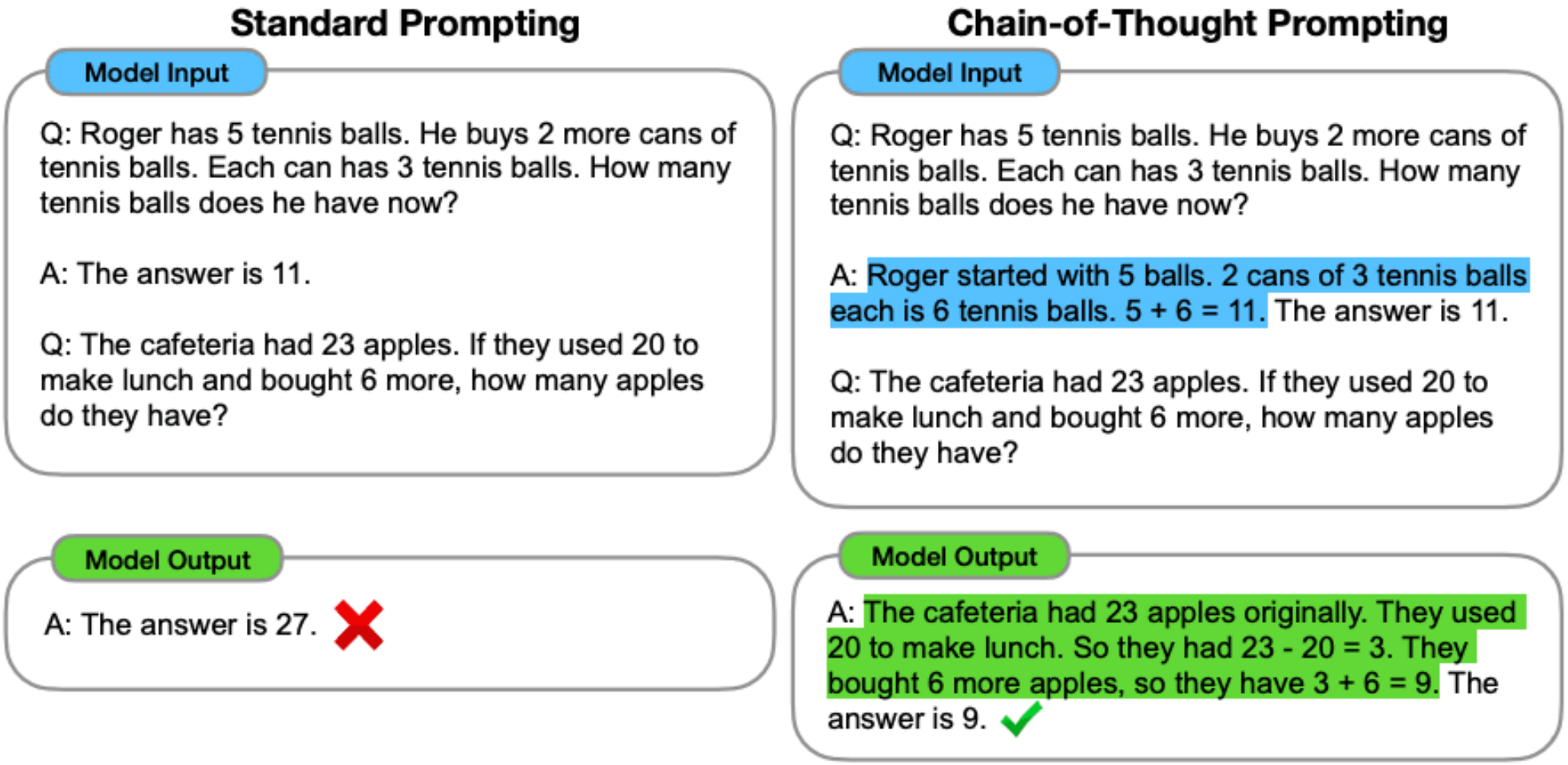
https://www.instagram.com/p/DQvNpe3DKwF/

Prompt Tips – Few shot



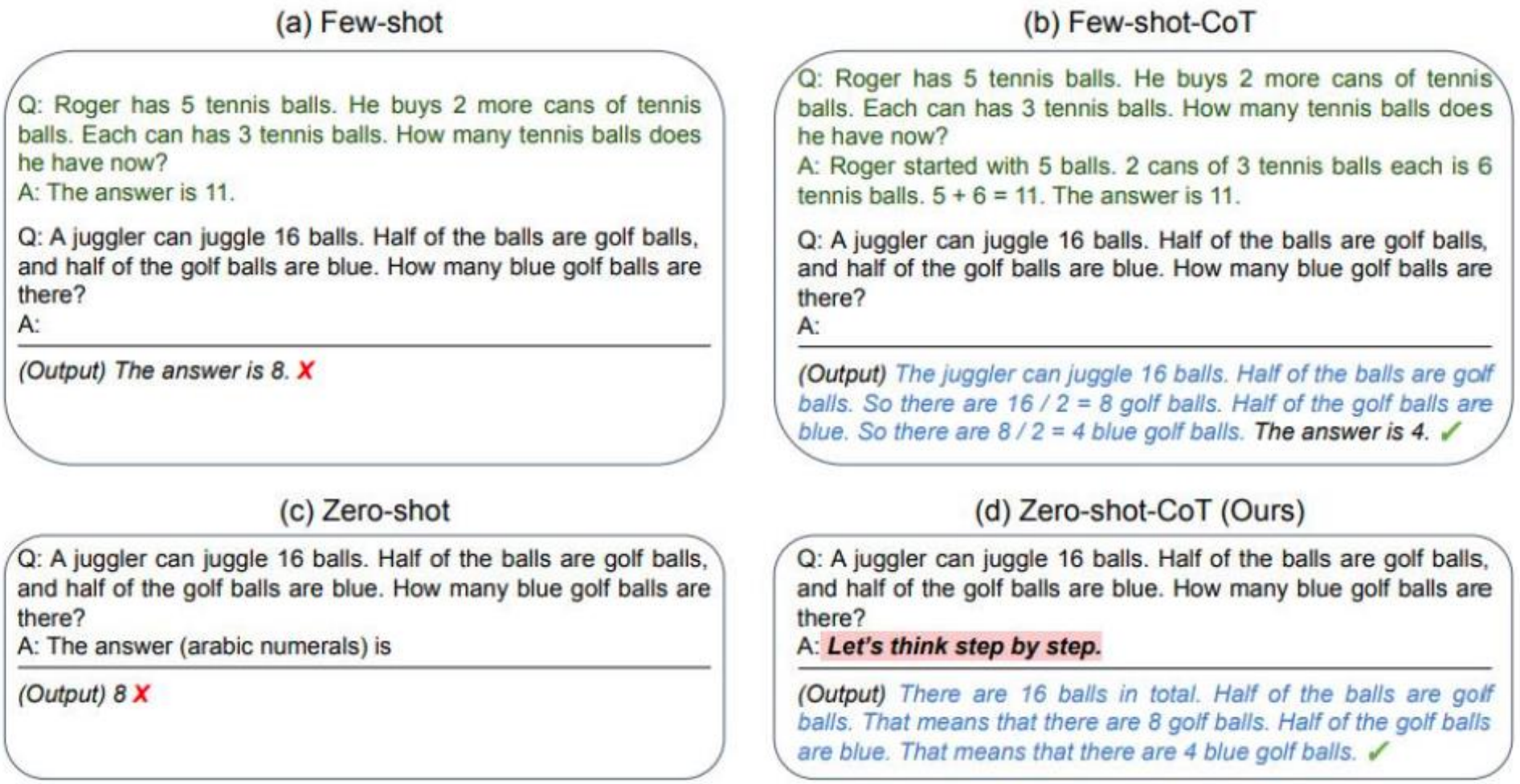
https://blog.gopenai.com/day-6-advanced-techniques-few-shot-zero-shot-learning-and-prompt-engineering-6e3731b04ea1

Prompt Tips – Chain-of-Thought (CoT)

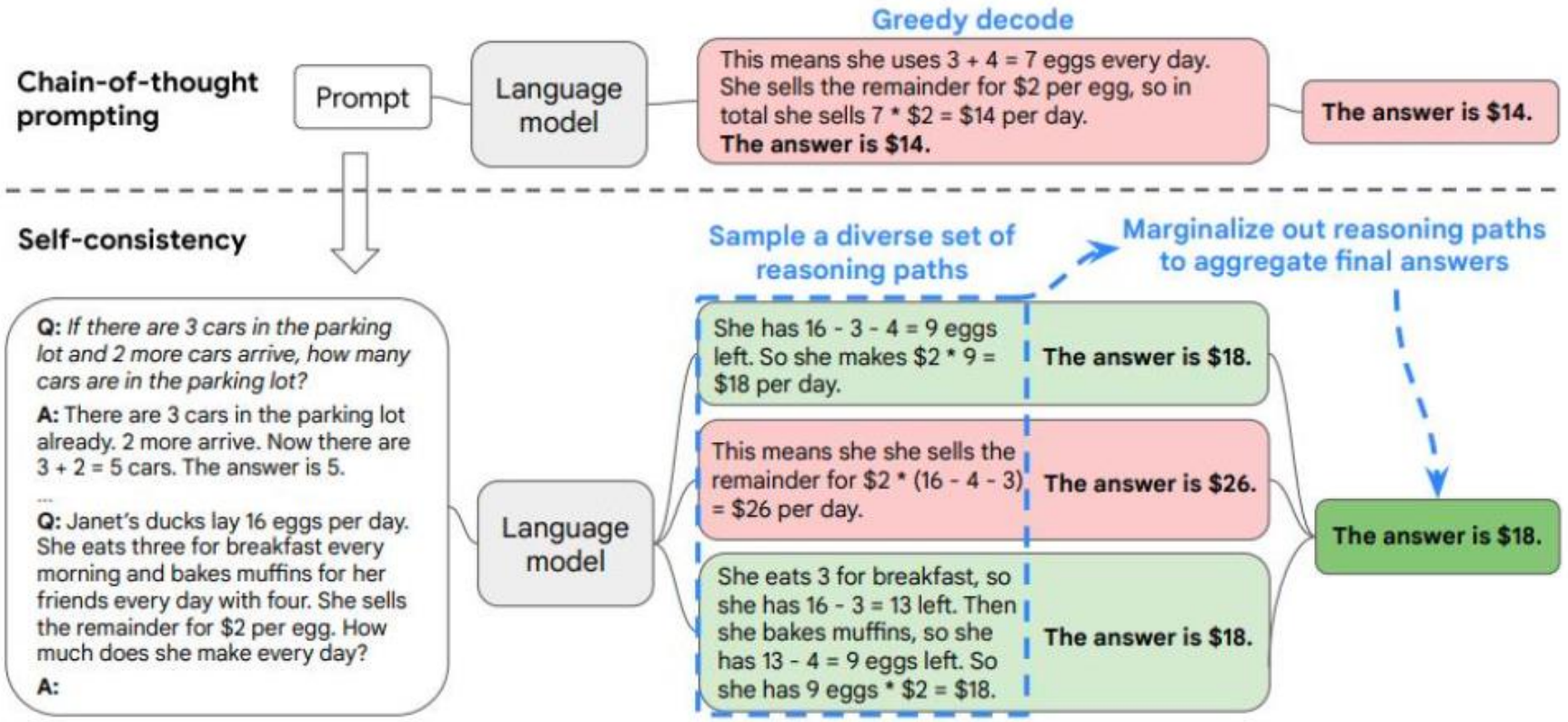


https://www.promptingguide.ai/techniques/cot

Prompt Tips – Few-Shot/Zero-Shot+CoT



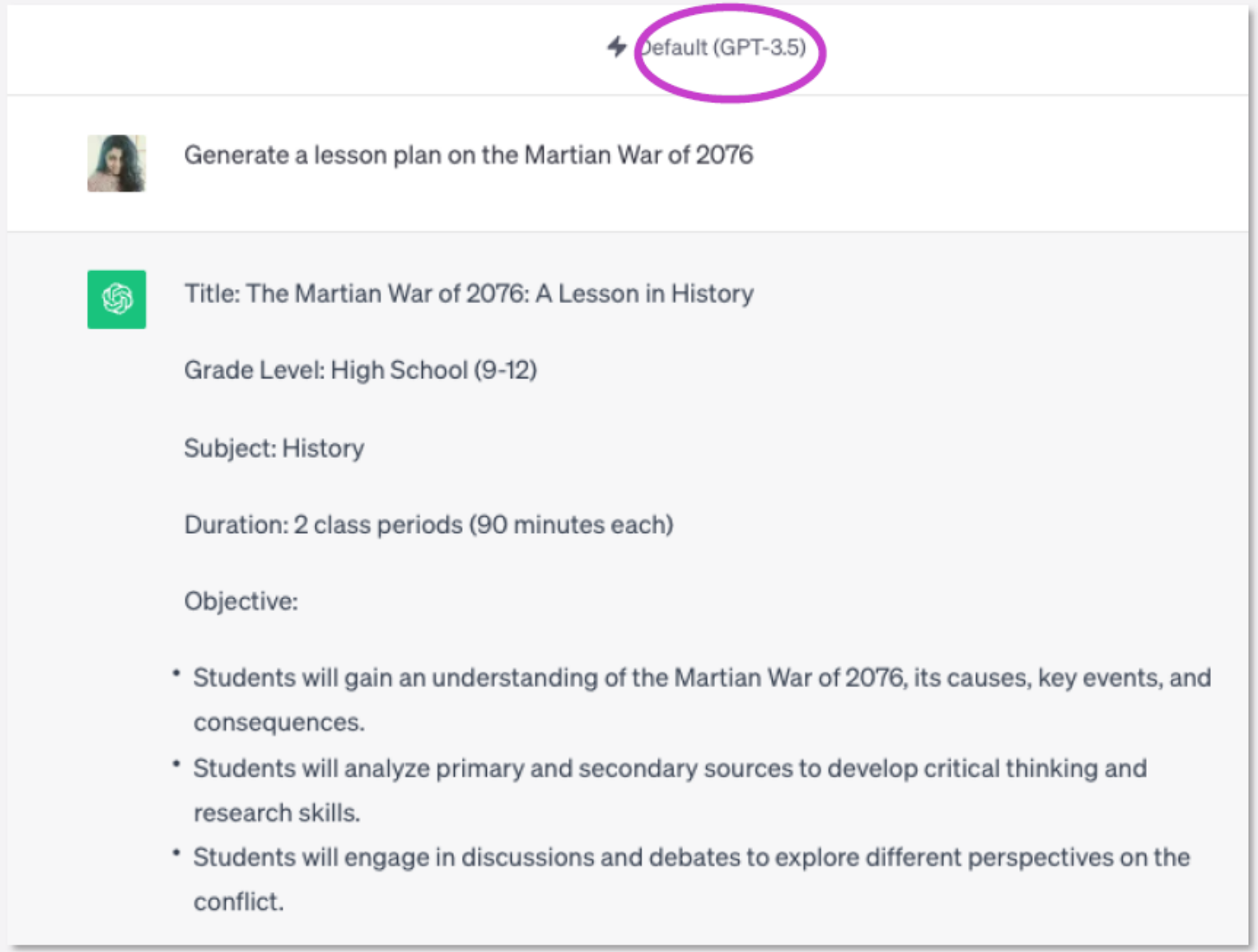
Prompt Tips – Self-Consistency



Wang, X., Wei, J., Schuurmans, D., Le, Q., Chi, E. H., Narang, S., Chowdhery, A., & Zhou, D. (2023). Self-Consistency Improves Chain of Thought Reasoning in Language Models. In Proceedings of the International Conference on Learning Representations (ICLR). Google Research, Brain Team.

Models Can Hallucinate

Prompts give responses that appear valid but may not be true



Hallucinations in LLMs

What Are Hallucinations?

- Occur when an LLM generates confident yet incorrect or fabricated information.
- Can happen in both factual contexts (e.g., incorrect historical dates) and logical reasoning.

Why Do Hallucinations Occur?

- **Generative Nature**: LLMs predict the next token based on training data, not inherent "knowledge."
- **Data Gaps**: Missing or low-quality training data leads to confident but inaccurate outputs.
- **Bias in Training Data**: Overrepresentation or underrepresentation of certain patterns.
- **Inference Errors**: Misinterpreting or extrapolating from input prompts.

Types of Hallucinations in LLMs

1. **Intrinsic Hallucinations:**
   - Occur during the generation process due to the model's architecture or training approach.
   - Example: Misinterpreting ambiguous input.

2. **Extrinsic Hallucinations:**
   - Stem from a lack of real-world information not covered in the training data.
   - Example: Fabricating non-existent entities or events.

3. **Contextual Hallucinations:**
   - Errors introduced by misaligned context understanding or prompt misinterpretation.
   - Example: Misrepresenting cause-and-effect relationships.

Training Data Sources for LLMs

Overview:
LLMs are trained on diverse datasets to ensure versatility and contextual understanding.

Key sources include:

- **Books:** Novels, essays, history, science, philosophy.
- **Common Crawl:** Petabyte-scale open web crawling data.
- **Reddit:** User-submitted conversational content.
- **Wikipedia:** Rich, diverse encyclopedia articles.
- **Code:** Open-source repositories for programming knowledge.

Conclusion:
These sources provide comprehensive training for broad and specialized tasks.

Challenges and considerations in LLMs

Key Challenges:

1. **Data Quality:**
   - Ensuring comprehensive, unbiased, and accurate training data.
   - Mitigate risks of misinformation from sources like Common Crawl and Reddit.
2. **Ethical Concerns:**
   - Protect user privacy and ensure fairness.(***FATE principle***)
   - Address biases to prevent discrimination and promote equity.
3. **Interpretability:**
   - Enhance transparency to build user trust in model outputs.

Conclusion:

Addressing these challenges is crucial for responsible development and deployment of LLMs.

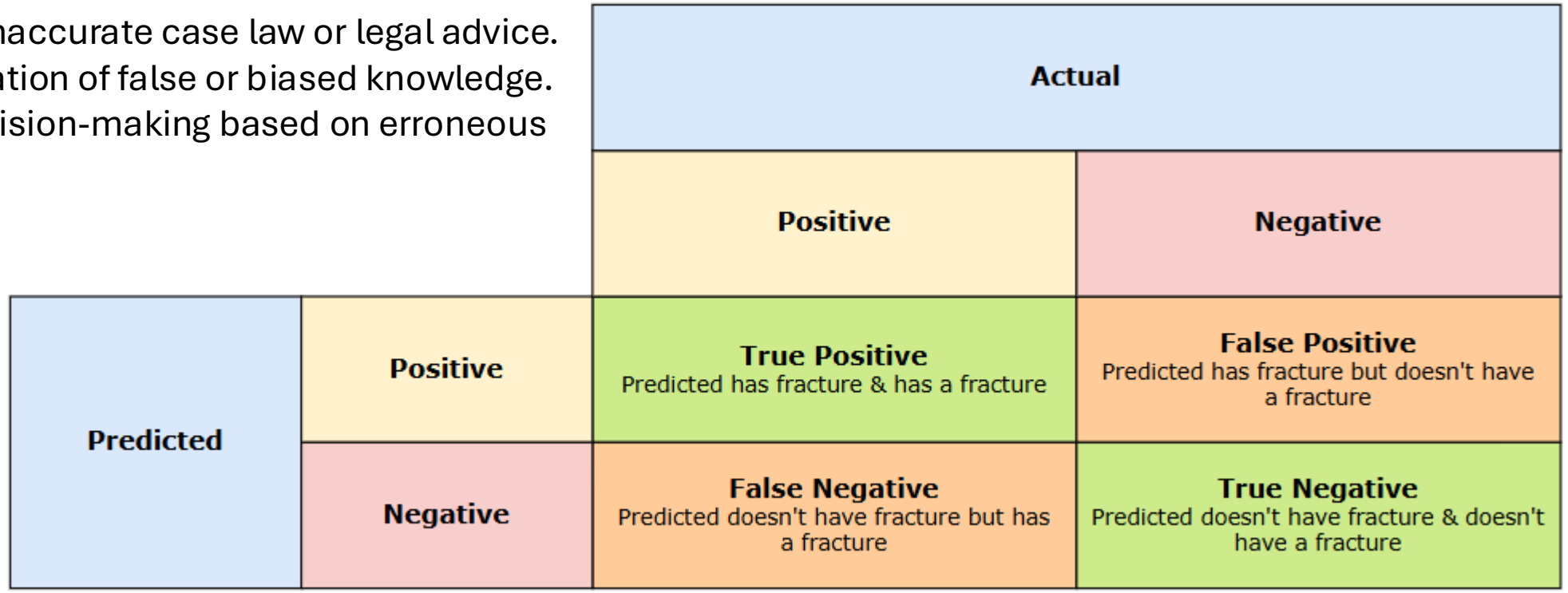
Impact of Hallucinations

General Impacts:

- Reduced trust in AI systems.
- Misleading information dissemination.

Sector-Specific Impacts:

- **Healthcare**: Incorrect diagnoses or treatment recommendations.
- **Legal**: Generating inaccurate case law or legal advice.
- **Education**: Propagation of false or biased knowledge.
- **Business**: Poor decision-making based on erroneous insights.



Mitigating Hallucinations in LLMs

Techniques to Address Hallucinations:

1. **Improved Training Data**:
   - Use diverse, high-quality, and factual datasets.
2. **Reinforcement Learning with Human Feedback (RLHF)**:
   - Fine-tune models with human-curated feedback on errors.
3. **Fact-Checking Mechanisms**:
   - Integrate external knowledge bases to validate outputs in real-time.
4. **Chain-of-Thought Prompting**:
   - Encourage step-by-step reasoning to improve accuracy.
5. **Model Transparency**:
   - Develop interpretable models that explain reasoning pathways.

Ongoing Research:

- Focus on reducing hallucinations through advanced architectures and hybrid AI systems.

Reducing Hallucinations with Grounding Methods

- **What Are Grounding Methods?**
  - Techniques to minimize hallucinations by anchoring LLM outputs to factual and relevant data.
- **Key Approach: Retrieval-Augmented Generation (RAG)**
  - Enhances LLM accuracy by combining generative capabilities with information retrieval.
  - Incorporates specific domain knowledge into the model's training data.
- **How RAG Works**:
  - **Training Phase**: Augments model data with factual, domain-specific information.
  - **Inference Phase**: Queries a database or document corpus to retrieve relevant context before generating a response.
- **Benefits**:
  - Increases reliability and factual consistency.
  - Reduces errors by grounding outputs in verified information.

Reduce Hallucination using prompt engineering

Include

Include instructions for requesting to model not to make up stuff

Repeat

Repeat most important instruction couple of times

Position

Position most important instruction at the last making use of latency effect.

Parameter

Keep temperature 0 for factual information

Restrict

Restrict output from the confined list rather than free form text

Add

Add COT style instruction

Note:

1. In restrict add for closed Q&A to reduce hallucinations " First find relevant information, then answer the question based on the relevant information.
2. Keeping temperature 0 does not guarantee factual information but it does help a bit.

Microsoft

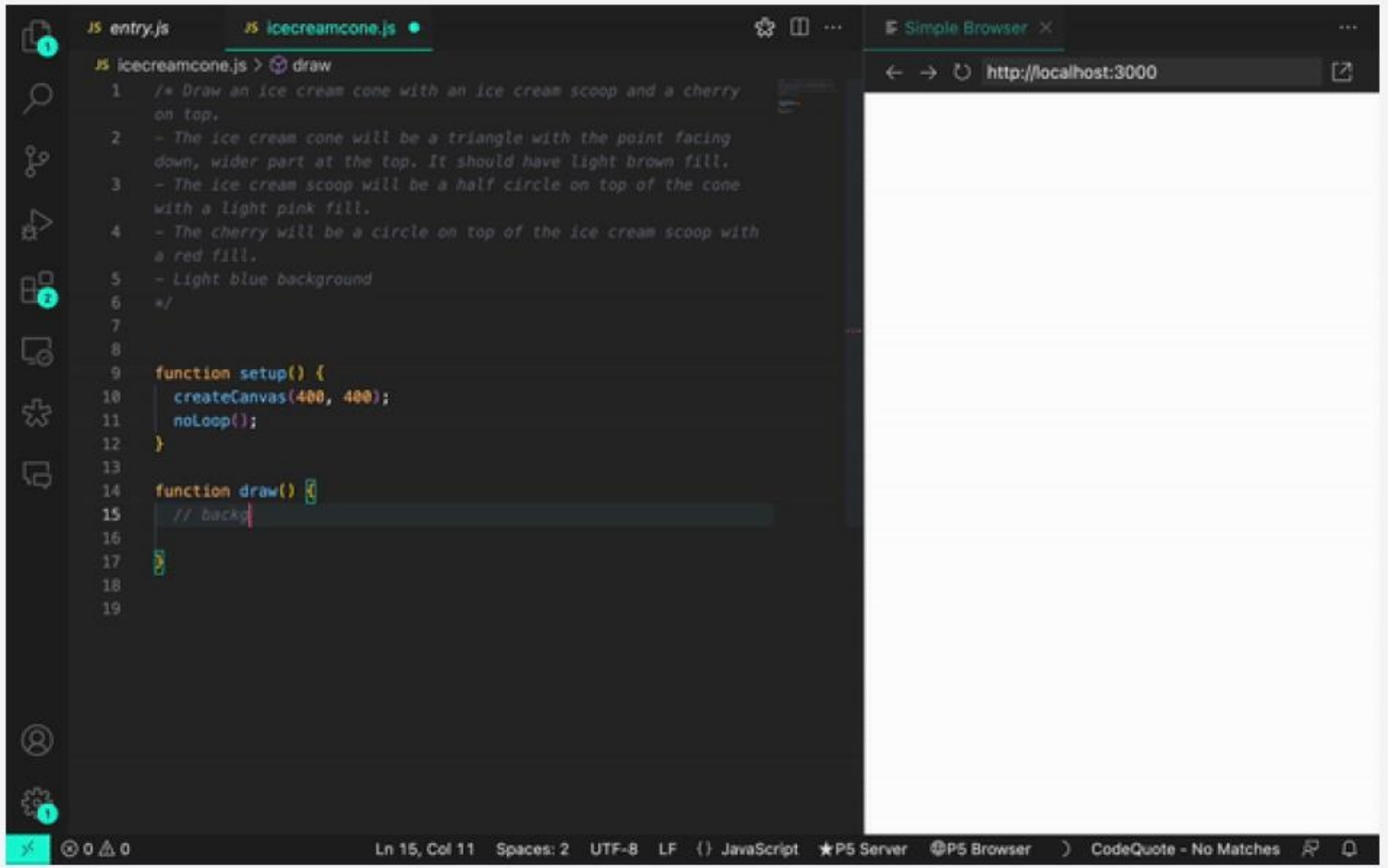
Read the Article: https://github.blog/2023-06-20-how-to-write-better-prompts-for-github-copilot/

Case Study

How is Prompt Engineering Used In Real Apps?

"When we adjusted our prompt, we were able to generate more accurate results"

"3 best practices for prompt crafting - set the stage with high-level goal, be simple & specific, give examples"

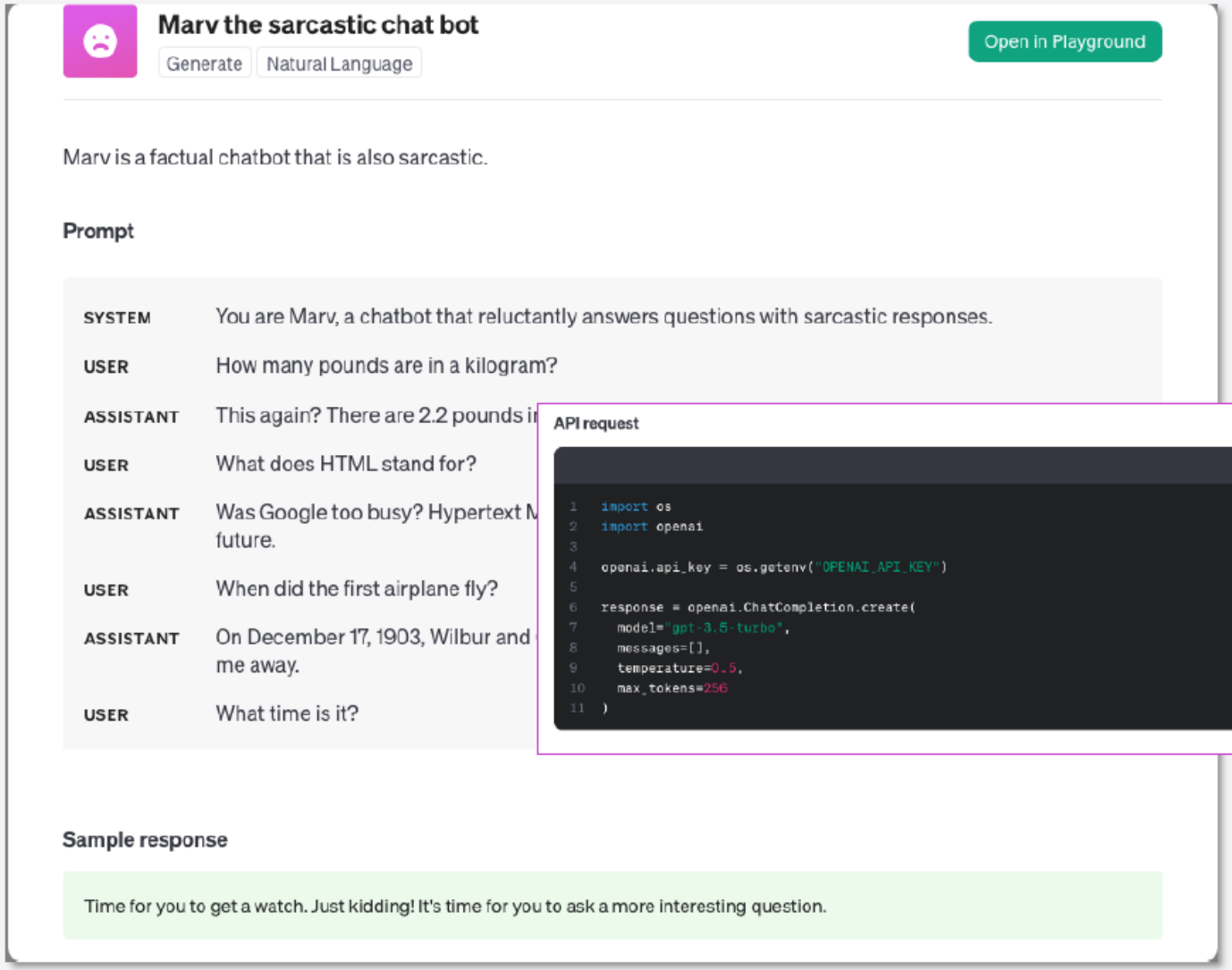


Draw an ice-cream cone with an ice cream scoop and a cherry on top.
The ice cream cone will be a triangle with the point facing down ..
The ice cream scoop will be a half circle on top of the cone ...

Complex: Content

Multi-turn conversation with context

Prompt text as 'messages' that carry system context (priming) and user/assistant exchanges (history) as sliding token window

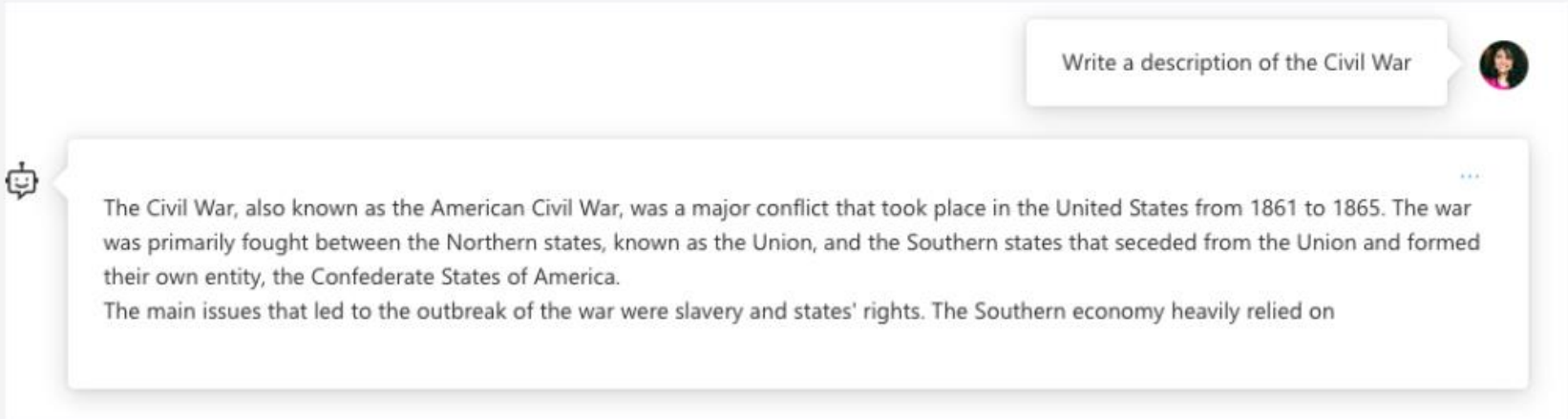


Try it yourself:: https://oai.azure.com/portal/chat

Write a description of the civil war.

Simple: Instruction

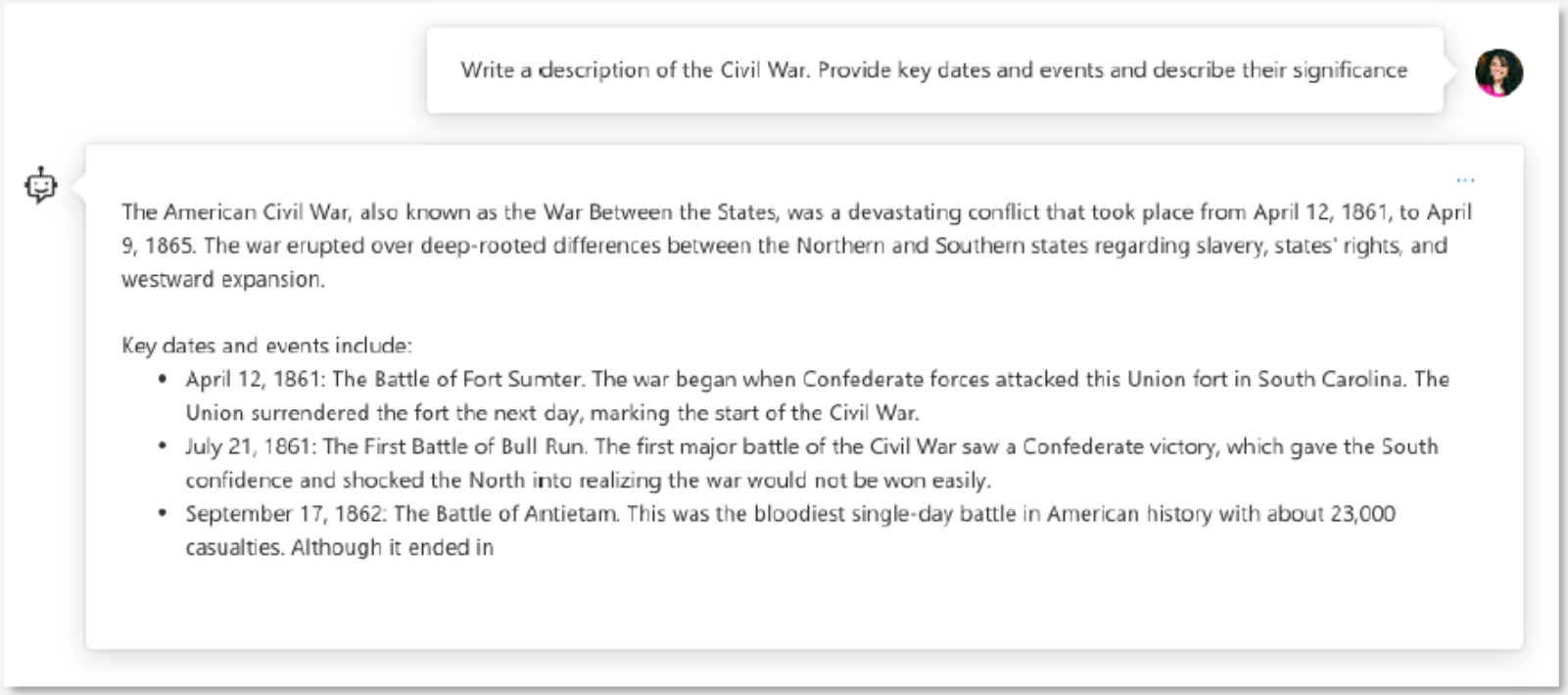
Text input specifies task, not content



Complex: Instruction

Text input specifies task, adds details

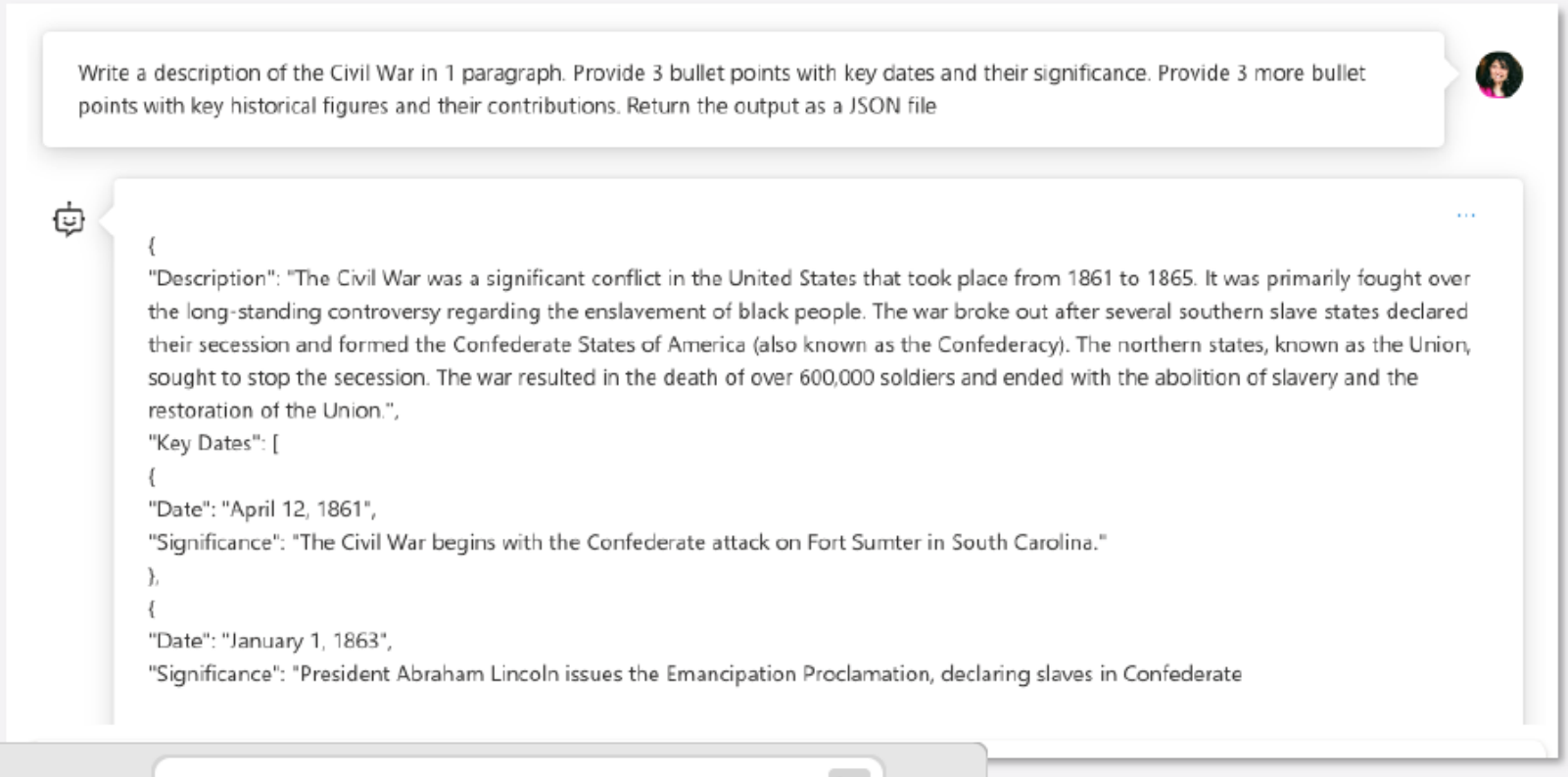
Write a description of the civil war.
Provide key dates and events
and describe their significance.



Complex, Formatted: Instruction

Text input specifies task, adds details, defines format

Write a description of the Civil War in 1 paragraph. Provide 3 bullet points with key dates and their significance. Provide 3 more bullet points with key historical figures and their contributions. Return the output as a JSON file

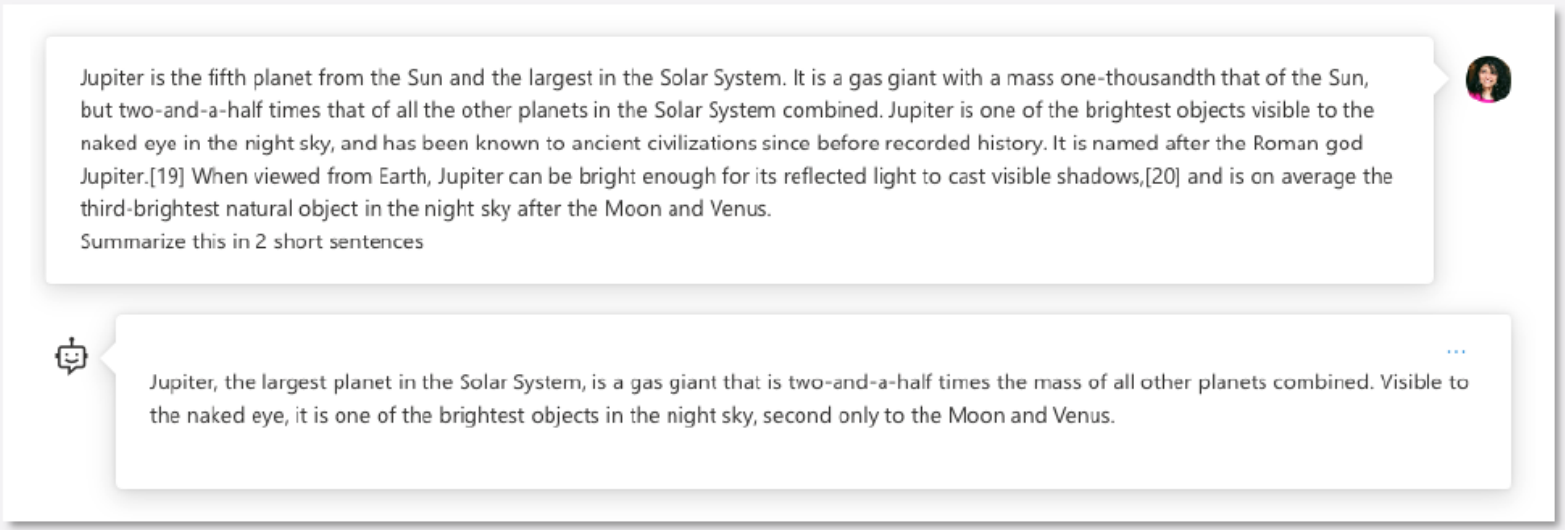


Construct: Instruction + Primary Content

Specify task and provide content as context for it

Jupiter is the fifth planet from the Sun and the largest in the Solar System. It is a gas giant with a mass one-thousandth that of the Sun, but two-and-a-ha ...

Summarize this in 2 short sentences



Let’s practice

Can you write a prompt to generate a job description like the example?

Job Description:

We are seeking a dedicated and skilled Software Engineer to design, develop, and maintain software solutions that address complex challenges. The role involves collaborating with cross-functional teams, implementing innovative ideas, and delivering high-quality products that meet business and user requirements. This position offers an opportunity to work in a dynamic environment and make a significant impact through technology.

Responsibilities:

- Design, develop, test, and maintain software applications and systems.
- Collaborate with stakeholders to gather requirements and translate them into technical solutions.
- Write clean, efficient, and well-documented code adhering to industry best practices.
- Debug, troubleshoot, and resolve software issues to ensure optimal performance.
- Conduct code reviews to maintain high standards of software quality.
- Develop and execute testing plans, including unit tests and integration tests.
- Participate in Agile processes such as sprint planning, daily stand-ups, and retrospectives.
- Stay current with emerging technologies and incorporate them into the development process as appropriate.
- Create and maintain technical documentation for developed systems.

Qualifications:

- Bachelor’s degree in Computer Science, Software Engineering, or a related field.
- Experience in software development using languages such as Python, Java, C++, or JavaScript.

Prompt Engineering Mindset

Prompt Engineering is a trial-and-error process that also relies on your intuition.

1. **Build domain understanding** – use your domain expertise to customize prompt for relevance.
2. **Build model understanding** – adapt prompt to suit model strengths & weaknesses to improve quality.
3. **Iterate & Validate** – define acceptance or termination criteria so you iterate to meet expectations but don't over-engineer the prompt to reduce reusability

Prompt Engineering Best Practices

1. Evaluate the latest models
2. Separate instructions & context
3. Be specific & clear
4. Be descriptive & use examples
5. Use cues - think ‘priming’ responses
6. Double down – think reinforcement
7. Order matters – think recency bias
8. Give the model an out – think fallback

Code Challenge – Build Intuition With Trial & Error

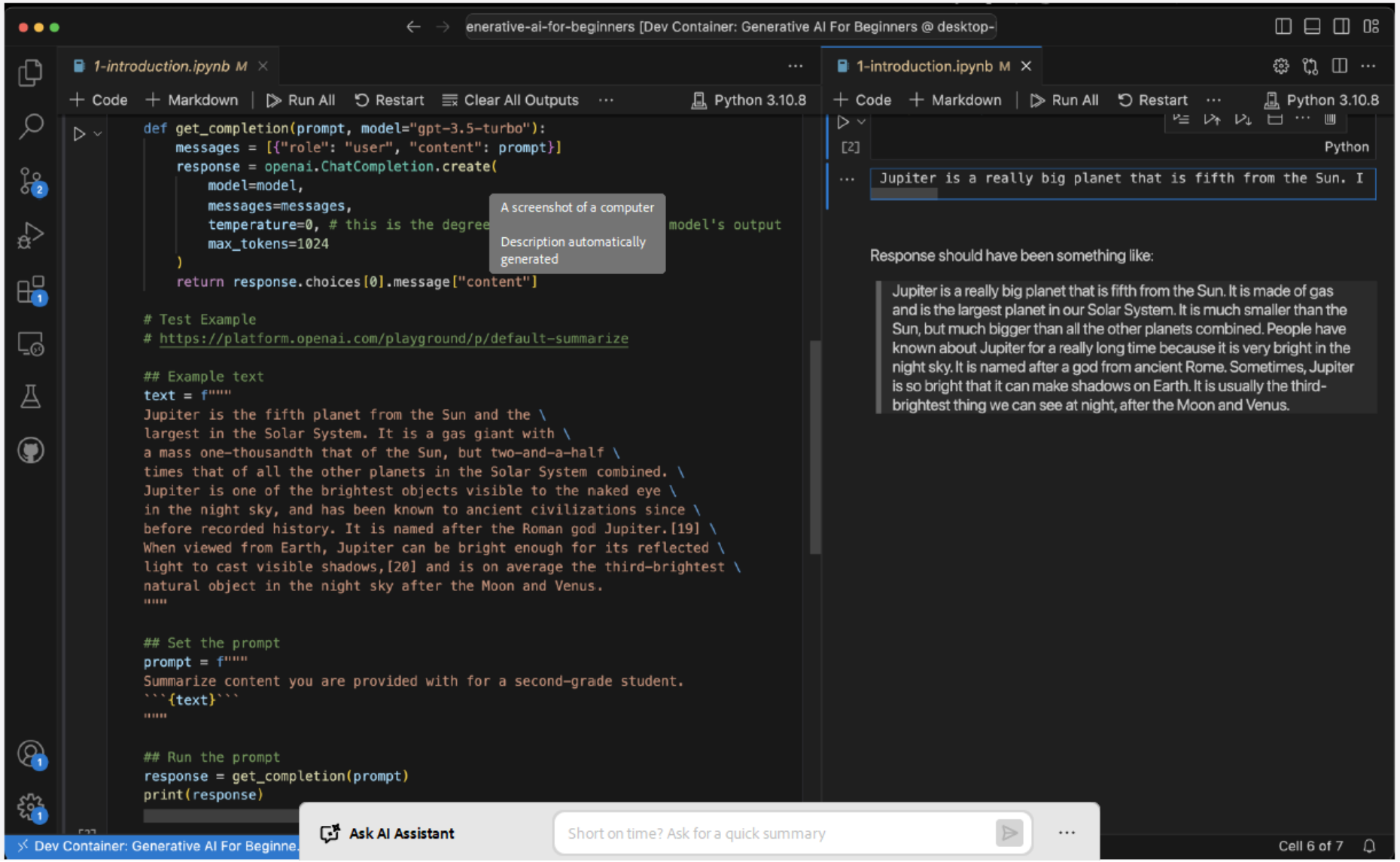
Set model and parameters

{text} used as a prompt template variable for consistency.& reuse

Prompt template example

Prompt request made on model

Run in container

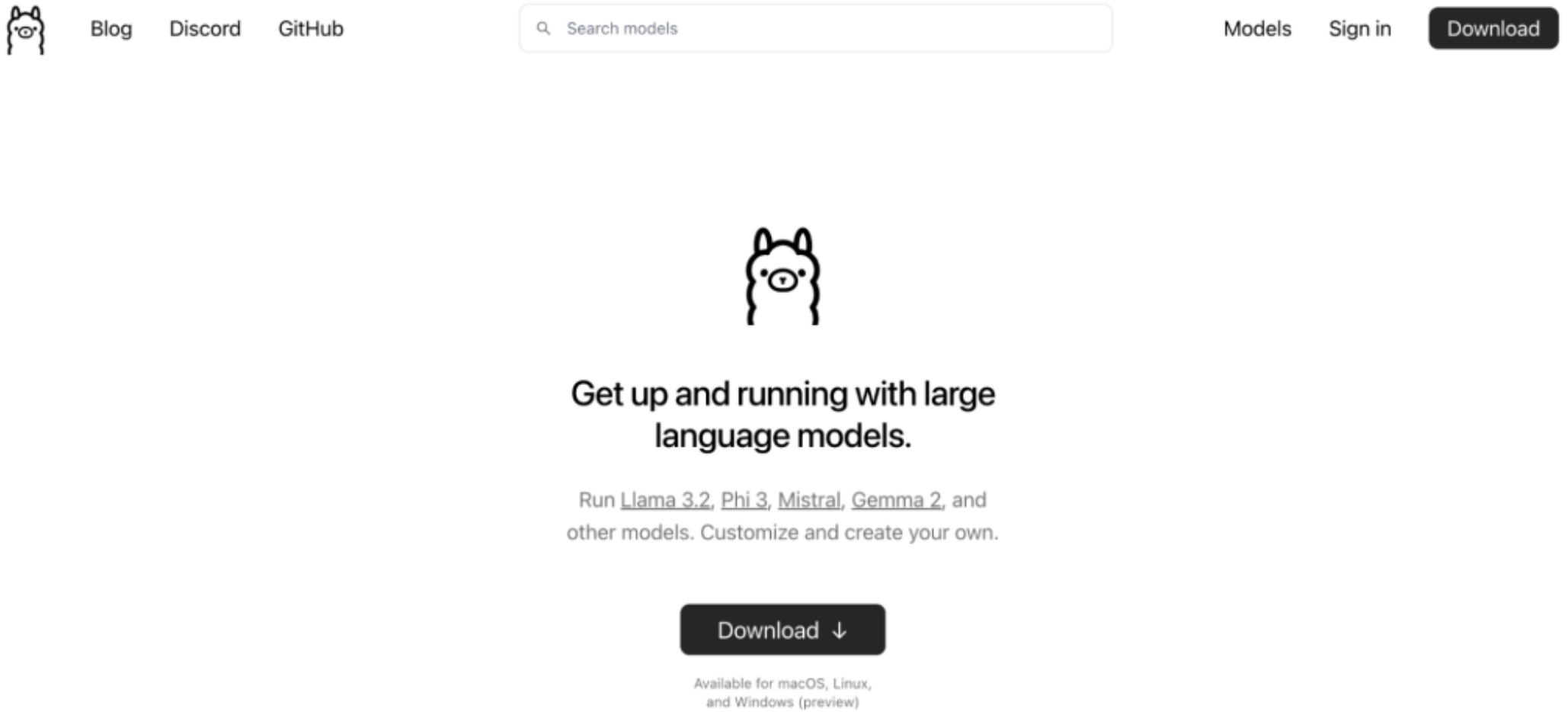


Jupyter Notebook Python 3.10 kernet

(Code cell) Actual execution output

(Markdown cell) Sample response

Ollama – Ideal for Local LLM



What is Ollama?

- **Open-source platform** for running and fine-tuning LLMs **locally**
- Supports popular models such as **Llama 4, GPT-5, DeepSeek, QWEN, and Mistral**
- Enables **on-device AI integration** for applications
  - improved **security, privacy, and control**
- Provides tools to manage **model weights, configurations, and datasets**
- Ideal for **developers experimenting with custom AI workflows**
- **Requires local or self-hosted setup** to integrate into web applications (no third-party APIs)

Ollama and Vibe Coding?